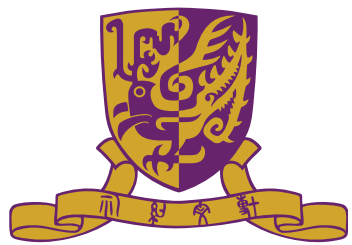
Machine Learning Introduction 3

Neural Networks

Yuning You

School of Science and Engineering
The Chinese University of Hong Kong, Shenzhen



Agenda

The Linear Models We Studied and Limitations

We have mainly studied linear regression and linear classification.

- Least squares (linear regression), perception, logistic regression, and SVM. They are all linear supervised machine learning models.
- Linear models do not perform well on highly non-linearly separable / regressible data.

The Linear Models We Studied and Limitations

We have mainly studied linear regression and linear classification.

- Least squares (linear regression), perception, logistic regression, and SVM. They are all linear supervised machine learning models.
- Linear models do not perform well on highly non-linearly separable / regressible data.

Many challenging objectives consist of non-linear structures. One way to deal with non-linear case is through nonlinear transformation of data, making the data to be linearly separable in higher dimension.

- Kernel method. However, this method needs to choose the right κ.

The Linear Models We Studied and Limitations

We have mainly studied linear regression and linear classification.

- Least squares (linear regression), perception, logistic regression, and SVM. They are all linear supervised machine learning models.
- Linear models do not perform well on highly non-linearly separable / regressible data.

Many challenging objectives consist of non-linear structures. One way to deal with non-linear case is through nonlinear transformation of data, making the data to be linearly separable in higher dimension.

- Kernel method. However, this method needs to choose the right κ.

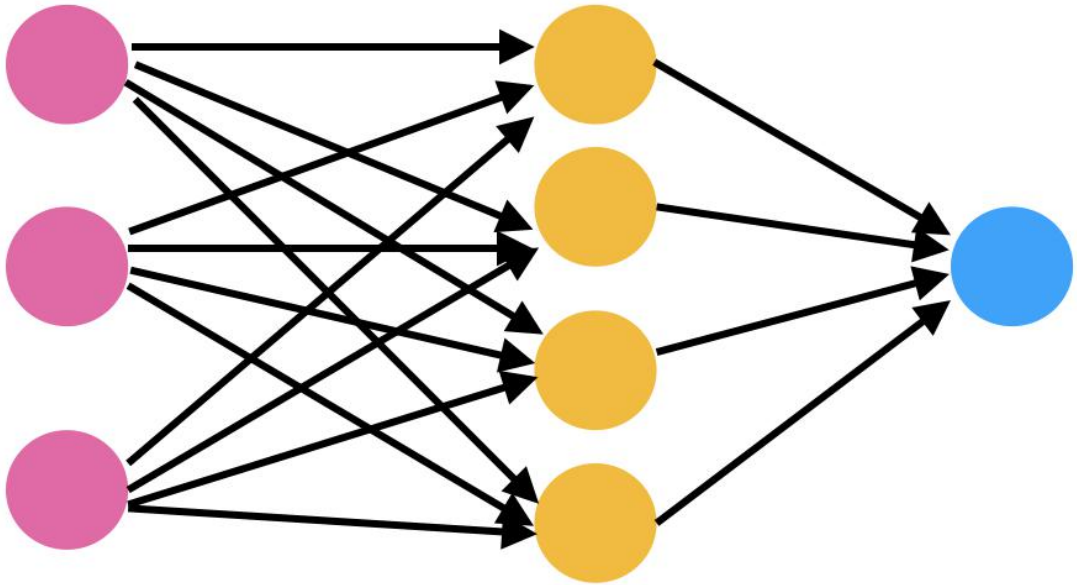
Another way is to impose non-linearity in our model $f_{\boldsymbol{\theta}}(\boldsymbol{x})$ (nonlinear in $\boldsymbol{\theta}$), leading to nonlinear supervised learning model.

- Note that supervised learning model is to use $f_{\boldsymbol{\theta}}(\boldsymbol{x})$ to approximate the underlying (nonlinear) pattern g.
- One most representative model is neural networks due to its universal approximation power.

$\rightsquigarrow$ Let us start with the basic model of neural networks.

Structure of Neural Network

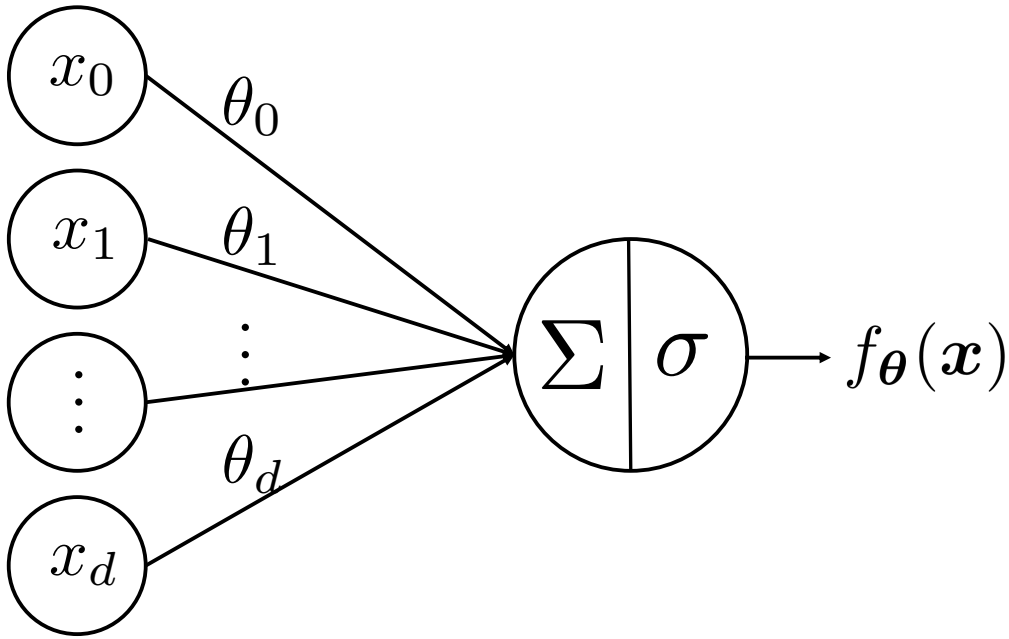
Neural network structure:



- Originally conceived as models for the brain.
  - Nodes are neurons.
  - Edges are synapses.

Let's start with the simplest single layer neural network.

Single Layer Neural Network

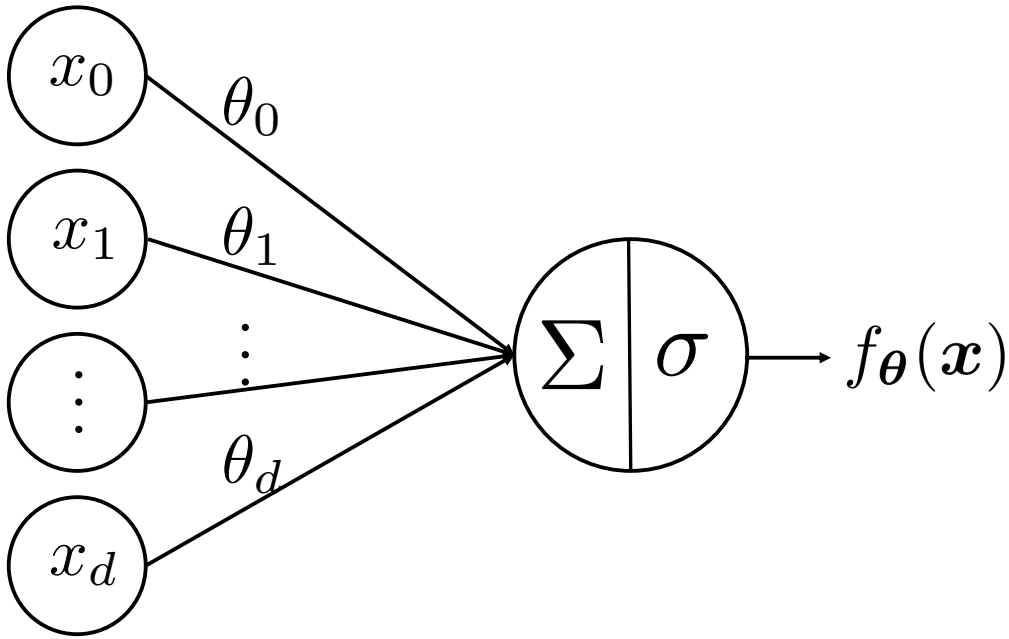


- Sample $\boldsymbol{x} = (x_0, \ldots, x_d)$ with $x_0 = 1$ (bias term).
- Parameters $\boldsymbol{\theta} = (\theta_0, \ldots, \theta_d)$, called weights.
- σ is called the activation function.

This neural network (NN) structure means

$$f_{\boldsymbol{\theta}}(\boldsymbol{x}) = \sigma\left(\sum_{i=0}^{d} \theta_i x_i\right) = \sigma(\boldsymbol{\theta}^\top \boldsymbol{x}).$$

Neural Network As A Generalized Linear Model



This single layer neural network covers linear models as special cases.

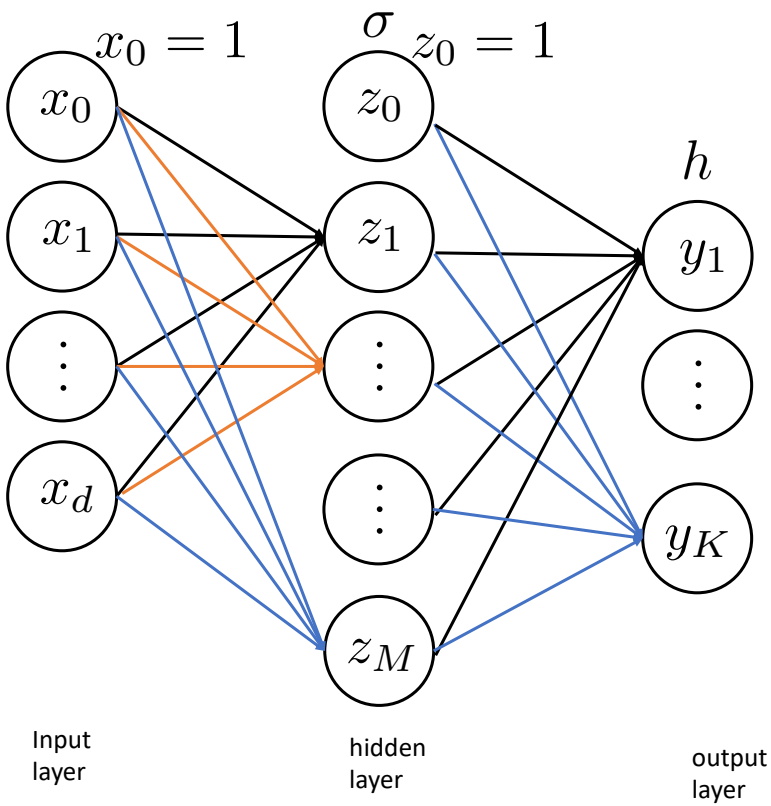
- Vanilla linear regression model if $\sigma(t) = t$.
- Logistic regression (binary case) if

$$\sigma(t) = \frac{1}{1 + e^{-y \cdot t}}$$

More generally, neural networks can have more than one layer.

One Hidden Layer (Two-Layer) Neural Network

We generalize neural network to two-layer:



Model:

$$z_m = \sigma(\boldsymbol{v}_m^\top \boldsymbol{x}), \quad m = 1, \ldots, M$$
$$y_k = h(\boldsymbol{w}_k^\top \boldsymbol{z}), \quad k = 1, \ldots, K$$

One Hidden Layer Neural Network

Mathematically, this one hidden layer neural network is represented as

$$z_m = \sigma(\boldsymbol{v}_m^\top \boldsymbol{x}), \quad m = 1, \ldots, M$$
$$y_k = h(\boldsymbol{w}_k^\top \boldsymbol{z}), \quad k = 1, \ldots, K$$

The main idea: Use the output of M linear models + possibly nonlinear transformation σ as the input to another linear model.

One Hidden Layer Neural Network

Mathematically, this one hidden layer neural network is represented as

$$z_m = \sigma(\boldsymbol{v}_m^\top \boldsymbol{x}), \quad m = 1, \ldots, M$$
$$y_k = h(\boldsymbol{w}_k^\top \boldsymbol{z}), \quad k = 1, \ldots, K$$

The main idea: Use the output of M linear models + possibly nonlinear transformation σ as the input to another linear model.

- σ, h are fixed activation functions.
- What are the unknown parameters?

One Hidden Layer Neural Network

Mathematically, this one hidden layer neural network is represented as

$$z_m = \sigma(\boldsymbol{v}_m^\top \boldsymbol{x}), \quad m = 1, \ldots, M$$
$$y_k = h(\boldsymbol{w}_k^\top \boldsymbol{z}), \quad k = 1, \ldots, K$$

The main idea: Use the output of M linear models + possibly nonlinear transformation σ as the input to another linear model.

- σ, h are fixed activation functions.
- What are the unknown parameters?

$$\boldsymbol{v}_m, \quad \boldsymbol{w}_k, \qquad m = 1, \ldots, M \quad \text{and} \quad k = 1, \ldots, K$$

One Hidden Layer Neural Network

Mathematically, this one hidden layer neural network is represented as

$$z_m = \sigma(\boldsymbol{v}_m^\top \boldsymbol{x}), \quad m = 1, \ldots, M$$
$$y_k = h(\boldsymbol{w}_k^\top \boldsymbol{z}), \quad k = 1, \ldots, K$$

The main idea: Use the output of M linear models + possibly nonlinear transformation σ as the input to another linear model.

- σ, h are fixed activation functions.
- What are the unknown parameters?

$$\boldsymbol{v}_m, \quad \boldsymbol{w}_k, \qquad m = 1, \ldots, M \quad \text{and} \quad k = 1, \ldots, K$$

- What are the dimension of $\boldsymbol{v}_m$ and $\boldsymbol{w}_k$?
- How many extra parameters compared to vanilla linear model?

Neural Network Model in Matrix Form

$$z_m = \sigma(\boldsymbol{v}_m^\top \boldsymbol{x}), \quad m = 1, \ldots, M.$$
$$y_k = h(\boldsymbol{w}_k^\top \boldsymbol{z}), \quad k = 1, \ldots, K.$$

Let us omit the offset/bias for simplicity and without loss of generality. Letting $\boldsymbol{V} \in \mathbb{R}^{M \times d}$ denote the matrix having rows $\boldsymbol{v}_m^\top$ and $\boldsymbol{W} \in \mathbb{R}^{K \times M}$ denote the matrix having rows $\boldsymbol{w}_k^\top$, we can write this neural net as

$$\boldsymbol{z} = \sigma(\boldsymbol{V}\boldsymbol{x}).$$
$$\boldsymbol{y} = h(\boldsymbol{W}\boldsymbol{z}).$$

One hidden layer NN model can be written compactly

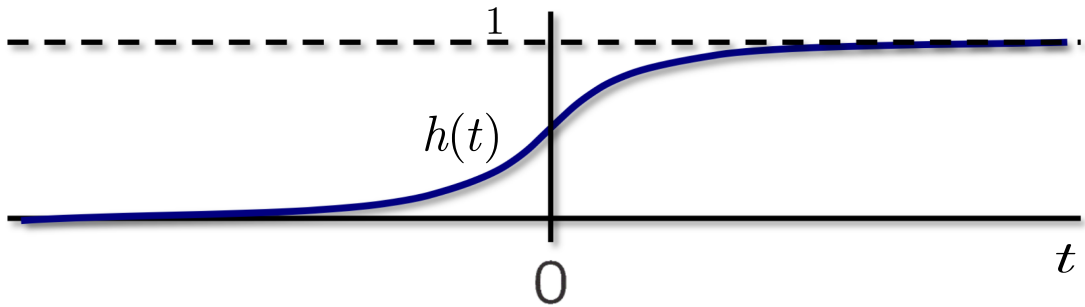
$$\boxed{\boldsymbol{y} = f_{\boldsymbol{\theta}}(\boldsymbol{x}) = h(\boldsymbol{W}\sigma(\boldsymbol{V}\boldsymbol{x})).}$$

- $\boldsymbol{\theta} := (\boldsymbol{V}, \boldsymbol{W})$ contains the network parameters, called weights.
- The input of the next layer is the output of the previous layer ($\boldsymbol{z} = \sigma(\boldsymbol{V}\boldsymbol{x})$).

The Ingredients in Typical Neural Networks

- A historically common choice for σ is the logistic / sigmoid function:

$$\sigma(t) = \frac{1}{1 + e^{-t}}.$$



- A similar choice to sigmoid activation is tanh activation:

$$\sigma(t) = \frac{e^{t} - e^{-t}}{e^{t} + e^{-t}}.$$

- Another quite popular choice of σ is the ReLU (rectified linear unit) activation:

$$\sigma(t) = \max(0, t).$$

The Ingredients in Typical Neural Networks

The choice of h depends somewhat on the application.

- Regression, we usually use

$$h(t) = t.$$

The Ingredients in Typical Neural Networks

The choice of h depends somewhat on the application.

- Regression, we usually use

$$h(t) = t.$$

- Logistic regression for binary classification, where $K = 1, y = \{+1, -1\}$. The predicted probability for the two classes:

The Ingredients in Typical Neural Networks

The choice of h depends somewhat on the application.

- Regression, we usually use

$$h(t) = t.$$

- Logistic regression for binary classification, where $K = 1, y = \{+1, -1\}$. The predicted probability for the two classes:

$$h(t) = \frac{1}{1 + e^{-y \cdot t}}.$$

  One can also use SVM model here.

The Ingredients in Typical Neural Networks

The choice of h depends somewhat on the application.

- Regression, we usually use

$$h(t) = t.$$

- Logistic regression for binary classification, where $K = 1, y = \{+1, -1\}$. The predicted probability for the two classes:

$$h(t) = \frac{1}{1 + e^{-y \cdot t}}.$$

  One can also use SVM model here.

- Multiclass classification

$$h(t_k) = \frac{e^{t_k}}{\sum_{j=1}^{K} e^{t_j}}, \quad t_k = \boldsymbol{w}_k^\top \boldsymbol{z},$$

  which is to let the last layer to be a multi-class logistic regression classifier.

$\rightsquigarrow$ h is to apply certain linear model ($\boldsymbol{z}$ as input) at the end of NN.

The Representation Power of Neural Networks

One natural question would be why consider neural network model for nonlinear case.

- Supervised learning is to approximate the underlying pattern g. The target function g is nonlinear in general.
- The following theorem shows a universal approximation power of just one hidden layer neural networks, which shows that neural networks can represent any regular function, and thus can approximate any underlying (possibly nonlinear) g.

Theorem: Universal approximation Power

Let g be a continuous function on a bounded subset of d-dimensional space. Then, there exists a one hidden layer neural network $f_{\boldsymbol{\theta}}$ with a finite number of hidden neurons that approximates g arbitrarily well. Namely, for all sample $\boldsymbol{x}$, we have $|f_{\boldsymbol{\theta}}(\boldsymbol{x}) - g(\boldsymbol{x})| < \varepsilon$ for every $\varepsilon > 0$.

Remarks

- Like kernel methods, neural networks fit a linear model in a nonlinear feature space.
- Unlike kernel methods, these nonlinear features are learned — i.e., the output of the last hidden layer.
- Interpretation of NN: One can think neural network model as extracting features by nonlinear network and finally putting the extracted feature $\boldsymbol{z}$ into the last layer for regression or classification. So, the last layer can be viewed as a linear model such as LS, LR, SVM, etc.
  $\rightsquigarrow$ This is also true for deep learning, i.e., deep neural network model (later).
- We will see that the training of neural network model involves nonconvex optimization because of the involved nonlinear structure of $f_{\boldsymbol{\theta}}$.

Agenda

Training Neural Networks

- Training data: $(\boldsymbol{x}_1, \boldsymbol{y}_1), \ldots, (\boldsymbol{x}_n, \boldsymbol{y}_n)$ with $\boldsymbol{x}_i \in \mathbb{R}^d$ and $\boldsymbol{y}_i \in \mathbb{R}^K$
- Neural network model

$$f_{\boldsymbol{\theta}}(\boldsymbol{x}) = h(\boldsymbol{W}\sigma(\boldsymbol{V}\boldsymbol{x})).$$

- We aim to learn the weight parameters $\boldsymbol{\theta} = (\boldsymbol{V}, \boldsymbol{W})$ such that

$$\boldsymbol{y}_i \leftarrow f_{\boldsymbol{\theta}}(\boldsymbol{x}_i).$$

  Thus, this is a supervised learning problem.

- We can quantify this approximation by choosing a loss function which we will seek to minimize by picking $\boldsymbol{\theta}$ appropriately

The Learning Problem for Training Neural Networks

Regression: $h(t) = t$ and use squared error (ℓ_2) loss, resulting in

- $K = 1$

$$\min_{\boldsymbol{\theta}} \left\{ \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^{n} (y_i - f_{\boldsymbol{\theta}}(\boldsymbol{x}_i))^2 \right\}$$

The Learning Problem for Training Neural Networks

Regression: $h(t) = t$ and use squared error (ℓ_2) loss, resulting in

- $K = 1$

$$\min_{\boldsymbol{\theta}} \left\{ \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^{n} (y_i - f_{\boldsymbol{\theta}}(\boldsymbol{x}_i))^2 \right\}$$

- $K > 1$

$$\min_{\boldsymbol{\theta}} \left\{ \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^{n} \|\boldsymbol{y}_i - f_{\boldsymbol{\theta}}(\boldsymbol{x}_i)\|_2^2 \right\}$$

The Learning Problem for Training Neural Networks

Classification (Binary): $K = 1, y = \{+1, -1\}$ and h is logistic function, and use logistic loss, resulting in

$$\min_{\boldsymbol{\theta}} \left\{ \mathcal{L}(\boldsymbol{\theta}) = -\frac{1}{n} \sum_{i=1}^{n} \log(f_{\boldsymbol{\theta}}(\boldsymbol{x}_i)) \right\}$$

The Learning Problem for Training Neural Networks

Classification (Binary): $K = 1, y = \{+1, -1\}$ and h is logistic function, and use logistic loss, resulting in

$$\min_{\boldsymbol{\theta}} \left\{ \mathcal{L}(\boldsymbol{\theta}) = -\frac{1}{n} \sum_{i=1}^{n} \log(f_{\boldsymbol{\theta}}(\boldsymbol{x}_i)) \right\}$$

Classification (Multi-class): $K > 1$, $\boldsymbol{y} = (0, \ldots, 1, \ldots, 0)$, h is softmax and use multiple-class logistic loss, resulting in

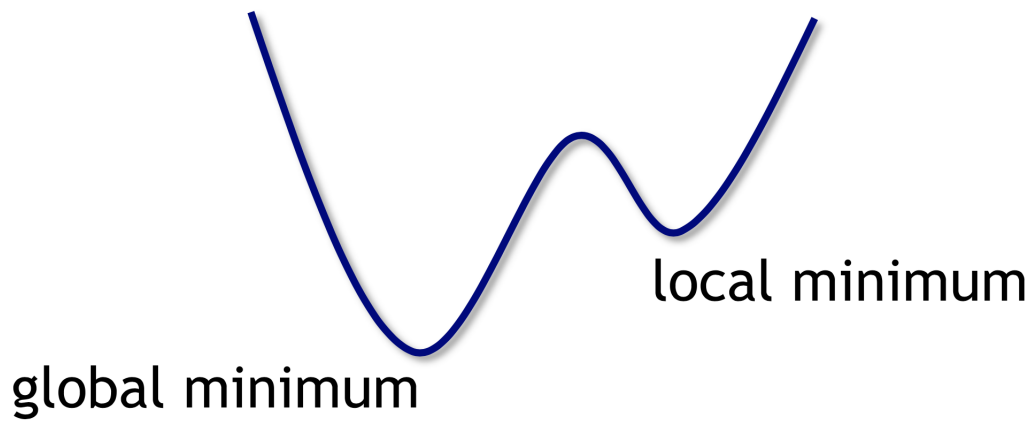
$$\min_{\boldsymbol{\theta}} \left\{ \mathcal{L}(\boldsymbol{\theta}) = -\frac{1}{n} \sum_{i=1}^{n} \boldsymbol{y}_i^\top \log(f_{\boldsymbol{\theta}}(\boldsymbol{x}_i)) \right\}$$

Training Neural Networks is Nonconvex Optimization

For example, regression training problem can be written as:

$$\min_{\boldsymbol{\theta}=(\boldsymbol{V},\boldsymbol{W})} \left\{ \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^{n} \|\boldsymbol{y}_i - h(\boldsymbol{W}\sigma(\boldsymbol{V}\boldsymbol{x}_i))\|^2 \right\}.$$

This is a highly nonconvex optimization problem.



Training Neural Networks

We put an abstract form of the former learning problems:

$$\min_{\boldsymbol{\theta}} \left\{ \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^{n} \ell_i(\boldsymbol{\theta}) \right\}$$

For different applications (regression or classification), ℓ_i has its own form.

- One can apply a gradient-based training algorithm.
- One needs to compute the gradient

$$\nabla \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^{n} \nabla \ell_i(\boldsymbol{\theta}).$$

- Apply gradient-based training algorithm:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \mu_k \nabla \mathcal{L}(\boldsymbol{\theta}_k).$$

However... The gradient is not easy to compute and GD training algorithm might be infeasible.

Agenda

Training Neural Networks

- Training data: $(\boldsymbol{x}_1, \boldsymbol{y}_1), \ldots, (\boldsymbol{x}_n, \boldsymbol{y}_n)$ with $\boldsymbol{x}_i \in \mathbb{R}^d$ and $\boldsymbol{y}_i \in \mathbb{R}^K$
- Neural network model

$$f_{\boldsymbol{\theta}}(\boldsymbol{x}) = h(\boldsymbol{W}\sigma(\boldsymbol{V}\boldsymbol{x})).$$

- We aim to learn the weight parameters $\boldsymbol{\theta} = (\boldsymbol{V}, \boldsymbol{W})$ such that

$$\boldsymbol{y}_i \leftarrow f_{\boldsymbol{\theta}}(\boldsymbol{x}_i).$$

  This is a supervised learning problem.

- We can quantify this approximation by choosing a loss function which we will seek to minimize by picking $\boldsymbol{\theta}$ appropriately

The Learning Problem for Training Neural Networks

Regression: $h(t) = t$ and use squared error (ℓ_2) loss, resulting in

- $K = 1$

$$\min_{\boldsymbol{\theta}} \left\{ \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^{n} (y_i - f_{\boldsymbol{\theta}}(\boldsymbol{x}_i))^2 \right\}.$$

The Learning Problem for Training Neural Networks

Regression: $h(t) = t$ and use squared error (ℓ_2) loss, resulting in

- $K = 1$

$$\min_{\boldsymbol{\theta}} \left\{ \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^{n} (y_i - f_{\boldsymbol{\theta}}(\boldsymbol{x}_i))^2 \right\}.$$

- $K > 1$

$$\min_{\boldsymbol{\theta}} \left\{ \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^{n} \|\boldsymbol{y}_i - f_{\boldsymbol{\theta}}(\boldsymbol{x}_i)\|_2^2 \right\}.$$

The Learning Problem for Training Neural Networks

Classification (Binary): $K = 1, y = \{+1, -1\}$ and h is logistic function. MLE principle leads to

$$\min_{\boldsymbol{\theta}} \left\{ \mathcal{L}(\boldsymbol{\theta}) = -\frac{1}{n} \sum_{i=1}^{n} \log(f_{\boldsymbol{\theta}}(\boldsymbol{x}_i)) \right\}.$$

The Learning Problem for Training Neural Networks

Classification (Binary): $K = 1, y = \{+1, -1\}$ and h is logistic function. MLE principle leads to

$$\min_{\boldsymbol{\theta}} \left\{ \mathcal{L}(\boldsymbol{\theta}) = -\frac{1}{n} \sum_{i=1}^{n} \log(f_{\boldsymbol{\theta}}(\boldsymbol{x}_i)) \right\}.$$

Classification (Multi-class): $K > 1$, $\boldsymbol{y} = (0, \ldots, 1, \ldots, 0)$, h is soft-max. MLE principle leads to

$$\min_{\boldsymbol{\theta}} \left\{ \mathcal{L}(\boldsymbol{\theta}) = -\frac{1}{n} \sum_{i=1}^{n} \boldsymbol{y}_i^\top \log(f_{\boldsymbol{\theta}}(\boldsymbol{x}_i)) \right\}.$$

The Learning Problem for Training Neural Networks

Classification (Binary): $K = 1, y = \{+1, -1\}$ and h is logistic function. MLE principle leads to

$$\min_{\boldsymbol{\theta}} \left\{ \mathcal{L}(\boldsymbol{\theta}) = -\frac{1}{n} \sum_{i=1}^{n} \log(f_{\boldsymbol{\theta}}(\boldsymbol{x}_i)) \right\}.$$

Classification (Multi-class): $K > 1$, $\boldsymbol{y} = (0, \ldots, 1, \ldots, 0)$, h is soft-max. MLE principle leads to

$$\min_{\boldsymbol{\theta}} \left\{ \mathcal{L}(\boldsymbol{\theta}) = -\frac{1}{n} \sum_{i=1}^{n} \boldsymbol{y}_i^\top \log(f_{\boldsymbol{\theta}}(\boldsymbol{x}_i)) \right\}.$$

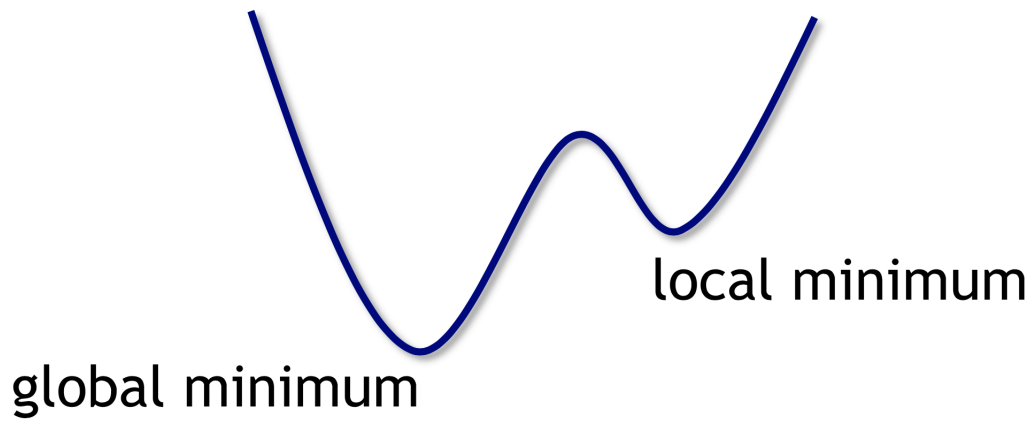
Summary: NN learning formulation is the same as least squares or logistic regression, with the difference being using $\boldsymbol{z} = \sigma(\boldsymbol{V}\boldsymbol{x})$ as input.

Training Neural Networks is Nonconvex Optimization

For example, regression training problem can be written as:

$$\min_{\boldsymbol{\theta}=(\boldsymbol{V},\boldsymbol{W})} \left\{ \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^{n} \|\boldsymbol{y}_i - h(\boldsymbol{W}\sigma(\boldsymbol{V}\boldsymbol{x}_i))\|^2 \right\}.$$

This is a highly nonconvex optimization problem.



Recall that linear supervised learning always gives rise to convex optimization. The nonconvexity of NN learning comes from learning the feature $\boldsymbol{z} = \sigma(\boldsymbol{V}\boldsymbol{x})$, where $\boldsymbol{V}$ is part of the learnable parameters.

Training Neural Networks

We put an abstract form of the former learning problems:

$$\min_{\boldsymbol{\theta}} \left\{ \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^{n} \ell_i(\boldsymbol{\theta}) \right\}$$

For different applications (regression or classification), ℓ_i has its own form.

Training Neural Networks

We put an abstract form of the former learning problems:

$$\min_{\boldsymbol{\theta}} \left\{ \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^{n} \ell_i(\boldsymbol{\theta}) \right\}$$

For different applications (regression or classification), ℓ_i has its own form.

- One can apply a gradient-based training algorithm.
- One needs to compute the gradient

$$\nabla \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^{n} \nabla \ell_i(\boldsymbol{\theta}).$$

- Apply gradient-based training algorithm:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \mu_k \nabla \mathcal{L}(\boldsymbol{\theta}_k).$$

However... The gradient is not easy to compute and GD training algorithm might be too optimistic.

Next: Training Algorithm and its Ingredients

We next dive into the depth of neural network training:

- How to compute the gradient?
  $\rightsquigarrow$ The well-known backpropagation (BP).
- In contemporary applications, n is so large. Applying GD is not feasible.
  $\rightsquigarrow$ Stochastic gradient descent, Adagrad, and Adam family.

Agenda

Computing The Gradient: Squared ℓ_2-Loss as An Example

- Let us take the regression case where we use squared error (ℓ_2) loss as an example. The conclusion applies to other cases.
- We consider the general case where $K > 1$. We have

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n}\sum_{i=1}^{n} \|\boldsymbol{y}_i - f_{\boldsymbol{\theta}}(\boldsymbol{x}_i)\|^2 = \frac{1}{n}\sum_{i=1}^{n} \ell_i(\boldsymbol{\theta})$$

  where

$$\ell_i(\boldsymbol{\theta}) = \|\boldsymbol{y}_i - f_{\boldsymbol{\theta}}(\boldsymbol{x}_i)\|_2^2$$

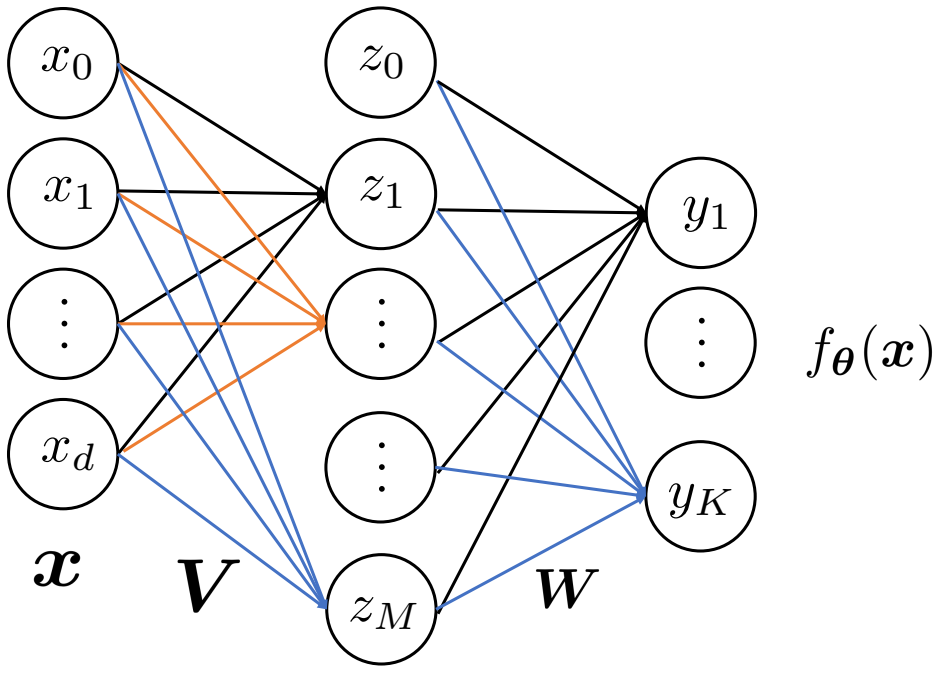
- Knowing how to take gradient for each ℓ_i suffices.

For ease of notation, we will omit the subscript i in ℓ_i and denote

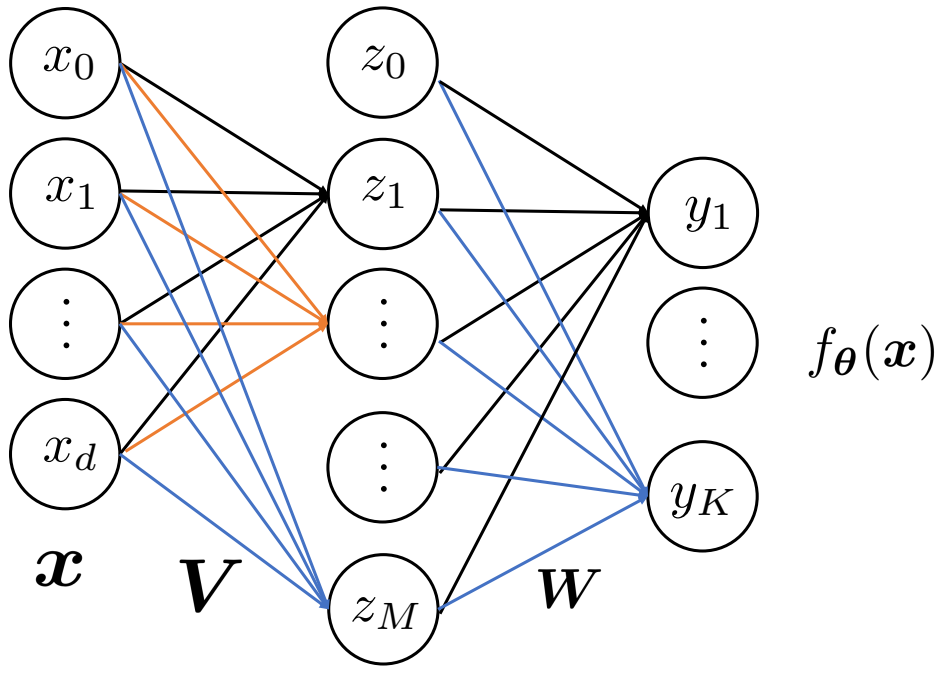
$$\ell(\boldsymbol{\theta}) = \|\boldsymbol{y} - f_{\boldsymbol{\theta}}(\boldsymbol{x})\|^2.$$

Task: Computing $\nabla \ell(\boldsymbol{\theta}) = (\frac{\partial \ell}{\partial \boldsymbol{V}}, \frac{\partial \ell}{\partial \boldsymbol{W}})$.

Backpropagation for Gradient Computation: Overview



Backpropagation for Gradient Computation: Overview



The idea: Computing gradient using chain rule:

- **Forward pass**: Given an $\boldsymbol{x}$, compute (using the current parameters $\boldsymbol{V}$ and $\boldsymbol{W}$) $\boldsymbol{z} = \sigma(\boldsymbol{V}\boldsymbol{x}), f_{\boldsymbol{\theta}}(\boldsymbol{x}) = h(\boldsymbol{W}\boldsymbol{z}), \ell(\boldsymbol{\theta}) = \|\boldsymbol{y} - f_{\boldsymbol{\theta}}(\boldsymbol{x})\|_2^2$
- **Backward pass**:
  - Second layer: compute $\frac{\partial \ell}{\partial \boldsymbol{W}}$, and compute $\frac{\partial \ell}{\partial \boldsymbol{z}}$.
  - First layer: Compute $\frac{\partial \ell}{\partial \boldsymbol{V}} = \frac{\partial \ell}{\partial \boldsymbol{z}} \times \frac{\partial \boldsymbol{z}}{\partial \boldsymbol{V}}$.

Deriving The Gradient: Second Layer

- We omit the offset w.l.o.g. i.e., no x_0 and z_0.
- By definition of gradient, we have

$$\frac{\partial \ell}{\partial \boldsymbol{W}} = \begin{bmatrix} \frac{\partial \ell}{\partial W(1,1)} & \cdots & \frac{\partial \ell}{\partial W(1,M)} \\ \vdots & \ddots & \vdots \\ \frac{\partial \ell}{\partial W(K,1)} & \cdots & \frac{\partial \ell}{\partial W(K,M)} \end{bmatrix} \in \mathbb{R}^{K \times M}.$$

- We focus on one element $\frac{\partial \ell}{\partial W(k,m)}$ of $\frac{\partial \ell}{\partial \boldsymbol{W}}$ for simplicity.
- Note that during the forward pass, we have computed

$$\ell(\boldsymbol{\theta}) = \|\boldsymbol{y} - h(\boldsymbol{W}\boldsymbol{z})\|_2^2,$$

where $\boldsymbol{z} = \sigma(\boldsymbol{V}\boldsymbol{x}) \in \mathbb{R}^M$.

Deriving The Gradient: Second Layer

$$
\begin{aligned}
\frac{\partial \ell}{\partial W(k,m)} &= \frac{\partial}{\partial W(k,m)} \|h(\boldsymbol{W}\boldsymbol{z}) - \boldsymbol{y}\|_2^2 \\
&= \frac{\partial}{\partial W(k,m)} \sum_{j=1}^{K} \left(h(\boldsymbol{w}_j^\top \boldsymbol{z}) - y[j]\right)^2 \\
&= \frac{\partial}{\partial W(k,m)} \left(h(\boldsymbol{w}_k^\top \boldsymbol{z}) - y[k]\right)^2 \qquad \text{using chain rule} \rightarrow \\
&= \frac{\partial \left(h(\boldsymbol{w}_k^\top \boldsymbol{z}) - y[k]\right)^2}{\partial \left(h(\boldsymbol{w}_k^\top \boldsymbol{z}) - y[k]\right)} \times \frac{\partial \left(h(\boldsymbol{w}_k^\top \boldsymbol{z}) - y[k]\right)}{\partial \boldsymbol{w}_k^\top \boldsymbol{z}} \times \frac{\partial \boldsymbol{w}_k^\top \boldsymbol{z}}{\partial W(k,m)} \\
&= 2\left(h(\boldsymbol{w}_k^\top \boldsymbol{z}) - y[k]\right) \times h'(\boldsymbol{w}_k^\top \boldsymbol{z}) \times z[m] \\
&:= \delta[k] \times z[m],
\end{aligned}
$$

where $\boldsymbol{w}_k^\top \in \mathbb{R}^M$ is the k-th row of $\boldsymbol{W}$ and we have defined

$$\delta[k] = 2\left(h(\boldsymbol{w}_k^\top \boldsymbol{z}) - y[k]\right) h'(\boldsymbol{w}_k^\top \boldsymbol{z}).$$

Deriving The Gradient: Second Layer

Since

$$\frac{\partial \ell}{\partial W(k,m)} = \delta[k] z[m]$$

Given $\delta[k] = 2\left(h(\boldsymbol{w}_k^\top \boldsymbol{z}) - y[k]\right) h'(\boldsymbol{w}_k^\top \boldsymbol{z})$, denote

$$\boldsymbol{\delta} = \begin{bmatrix} \delta[1] \\ \delta[2] \\ \vdots \\ \delta[K] \end{bmatrix} = 2(h(\boldsymbol{W}\boldsymbol{z}) - \boldsymbol{y}) \odot h'(\boldsymbol{W}\boldsymbol{z}) \in \mathbb{R}^K$$

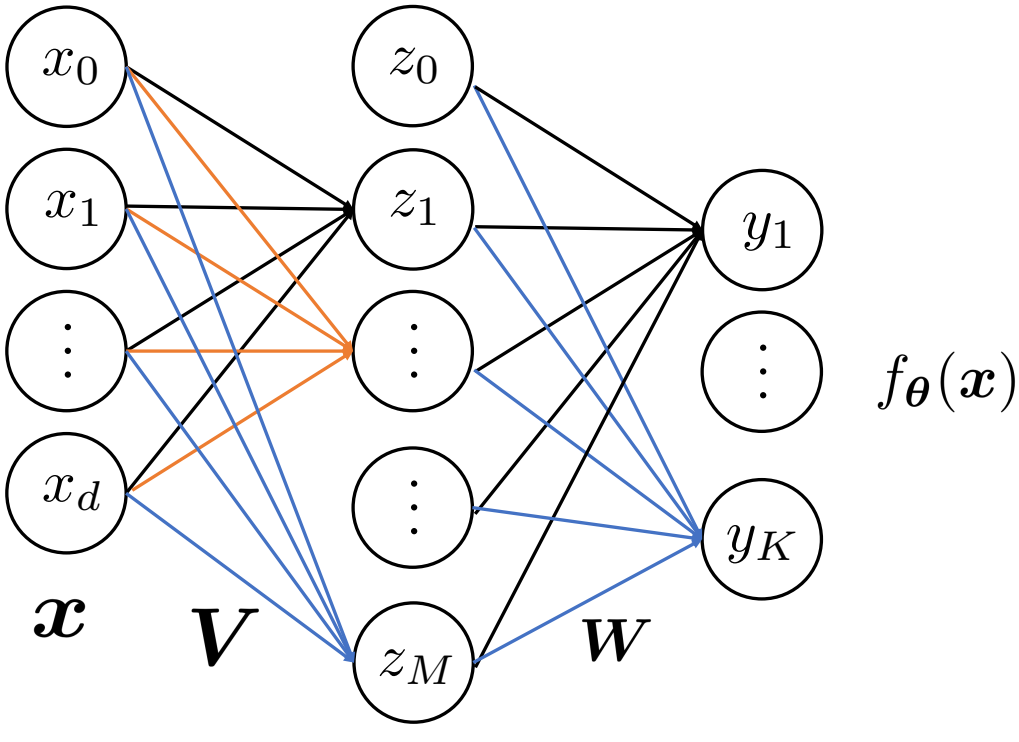
where $\odot$ is the Hadamard (element-wise) product.

We have

$$\frac{\partial \ell}{\partial \boldsymbol{W}} = \underbrace{\boldsymbol{\delta}}_{K\times 1} \underbrace{\boldsymbol{z}^\top}_{1\times M} \in \mathbb{R}^{K\times M},$$

which can be calculated using the current values of $\boldsymbol{W}$ and $\boldsymbol{z}$.

Deriving The Gradient: First Layer



$$\ell(\boldsymbol{\theta}) = \|\boldsymbol{y} - f_{\boldsymbol{\theta}}(\boldsymbol{x})\|_2^2, f_{\boldsymbol{\theta}}(\boldsymbol{x}) = h(\boldsymbol{W}\boldsymbol{z}), \boldsymbol{z} = \sigma(\boldsymbol{V}\boldsymbol{x})$$

We firstly back-propagate the gradient back to $\boldsymbol{z}$, and then calculate the gradient of $\boldsymbol{z}$ w.r.t. $\boldsymbol{V}$

$$\frac{\partial \ell}{\partial \boldsymbol{V}} = \frac{\partial \ell}{\partial \boldsymbol{z}} \times \frac{\partial \boldsymbol{z}}{\partial \boldsymbol{V}}$$

Deriving The Gradient: First Layer

To begin with, we back-propagate the gradient back to $\boldsymbol{z}$.
For an individual node of $\boldsymbol{z}$:

$$\begin{aligned}
\frac{\partial \ell}{\partial \boldsymbol{z}[m]} &= \frac{\partial}{\partial \boldsymbol{z}[m]} \|h(\boldsymbol{W}\boldsymbol{z}) - \boldsymbol{y}\|_2^2 \\
&= \frac{\partial}{\partial \boldsymbol{z}[m]} \sum_{k=1}^{K} \left(h(\boldsymbol{w}_k^\top \boldsymbol{z}) - y[k]\right)^2 \quad \text{using chain rule} \rightarrow \\
&= \sum_{k=1}^{K} 2\left(h(\boldsymbol{w}_k^\top \boldsymbol{z}) - y[k]\right) \times h'(\boldsymbol{w}_k^\top \boldsymbol{z}) \times w_k[m] \\
&= \sum_{k=1}^{K} \delta[k] w_k[m]
\end{aligned}$$

Combining the individual nodes of $\boldsymbol{z}$, we denote

$$\frac{\partial \ell}{\partial \boldsymbol{z}} = \boldsymbol{W}^\top \boldsymbol{\delta}$$

Deriving The Gradient: First Layer

$$\frac{\partial \ell}{\partial \boldsymbol{V}} = \begin{bmatrix} \frac{\partial \ell}{\partial V(1,1)} & \cdots & \frac{\partial \ell}{\partial V(1,d)} \\ \vdots & \ddots & \vdots \\ \frac{\partial \ell}{\partial V(M,1)} & \cdots & \frac{\partial \ell}{\partial V(M,d)} \end{bmatrix} \in \mathbb{R}^{M \times d}.$$

For an individual element of $\boldsymbol{V}$:

$$\frac{\partial \ell}{\partial V(m,j)} = \underbrace{\frac{\partial \ell}{\partial \boldsymbol{z}[m]}}_{\text{gradient from next layer}} \times \underbrace{\frac{\partial \boldsymbol{z}[m]}{\partial V(m,j)}}_{\text{local gradient}} \qquad \text{(chain rule)}$$

$$= \frac{\partial \ell}{\partial \boldsymbol{z}[m]} \times \frac{\partial \sigma(\boldsymbol{v}_m^\top \boldsymbol{x})}{\partial V(m,j)} \qquad (\text{since } \boldsymbol{z} = \sigma(\boldsymbol{V}\boldsymbol{x}))$$

$$= \sum_{k=1}^{K} \delta[k] w_k[m] \times \sigma'(\boldsymbol{v}_m^\top \boldsymbol{x}) \times x[j]$$

Combining the gradients of the individual elements:

$$\frac{\partial \ell}{\partial \boldsymbol{V}} = \underbrace{\left((\boldsymbol{W}^\top \boldsymbol{\delta}) \odot \sigma'(\boldsymbol{V}\boldsymbol{x}) \right)}_{M \times 1} \times \underbrace{\boldsymbol{x}^\top}_{1 \times d} \in \mathbb{R}^{M \times d}$$

The Overall Procedure

- Recall that for each data $\boldsymbol{x}_i$, we need to compute its corresponding gradient $\nabla \ell_i(\boldsymbol{\theta})$.
- Since $\boldsymbol{\theta} = (\boldsymbol{V}, \boldsymbol{W})$, we have

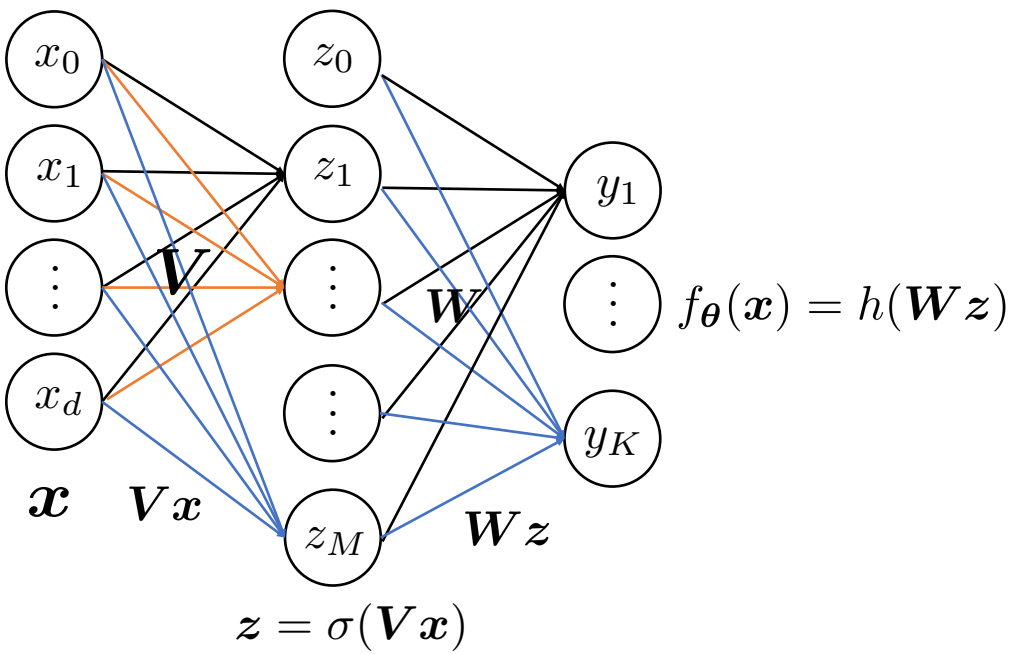
$$\nabla \ell_i(\boldsymbol{\theta}) = \left(\frac{\partial \ell_i}{\partial \boldsymbol{V}}, \frac{\partial \ell_i}{\partial \boldsymbol{W}} \right)$$

- This is achieved by an efficient two-pass method for computing the gradient at each iteration—the so-called backpropagation:
  1. Forward pass
  2. Backward pass

Backpropagation: Forward pass

Forward pass: Feed $\boldsymbol{x}_i$ into the network, use the current parameters $\boldsymbol{V}$ and $\boldsymbol{W}$ to compute and store

- $\boldsymbol{V}\boldsymbol{x}_i$
- $\boldsymbol{z}_i = \sigma(\boldsymbol{V}\boldsymbol{x}_i)$
- $\boldsymbol{W}\boldsymbol{z}_i$
- $f_{\boldsymbol{\theta}}(\boldsymbol{x}_i) = h(\boldsymbol{W}\boldsymbol{z}_i)$, $\ell_i(\boldsymbol{\theta}) = \|\boldsymbol{y} - f_{\boldsymbol{\theta}}(\boldsymbol{x}_i)\|_2^2$



Backpropagation: Backward pass

Backward pass:

Compute the gradient of the second layer

- $\boldsymbol{\delta}_i = 2(h(\boldsymbol{W}\boldsymbol{z}_i) - \boldsymbol{y}_i) \odot h'(\boldsymbol{W}\boldsymbol{z}_i)$
- $\frac{\partial \ell_i}{\partial \boldsymbol{W}} = \boldsymbol{\delta}_i \boldsymbol{z}_i^\top$

Backpropagate the gradient to $\boldsymbol{z}_i$:

- $\frac{\partial \ell}{\partial \boldsymbol{z}_i} = \boldsymbol{W}^\top \boldsymbol{\delta}_i$

Compute the gradient of the first layer:

- $\frac{\partial \ell_i}{\partial \boldsymbol{V}} = \left(\frac{\partial \ell}{\partial \boldsymbol{z}_i} \odot \sigma'(\boldsymbol{V}\boldsymbol{x}_i) \right) \boldsymbol{x}_i^\top$

Thus, backpropagation is an efficient way of computing the gradient of NN learning problem.

Given the computed gradient, we can apply gradient-based training algorithm for training NN. $\rightsquigarrow$ Which algorithm should we choose?

Structured Learning Problem: Finite-Sum Structure

In LS, LR, SVM, and NN, we always have

$$\min_{\boldsymbol{\theta}\in\mathbb{R}^d} \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n}\sum_{i=1}^{n} \underbrace{\ell(\boldsymbol{\theta}; \boldsymbol{x}_i, y_i)}_{\ell_i(\boldsymbol{\theta})}$$

each $\ell_i(\boldsymbol{\theta})$ is called a component function. We can use BP to compute $\nabla\mathcal{L}(\boldsymbol{\theta})$ if it is NN.

Structured Learning Problem: Finite-Sum Structure

In LS, LR, SVM, and NN, we always have

$$\min_{\boldsymbol{\theta}\in\mathbb{R}^d} \ \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n}\sum_{i=1}^{n} \underbrace{\ell(\boldsymbol{\theta}; \boldsymbol{x}_i, y_i)}_{\ell_i(\boldsymbol{\theta})}$$

each $\ell_i(\boldsymbol{\theta})$ is called a component function. We can use BP to compute $\nabla\mathcal{L}(\boldsymbol{\theta})$ if it is NN.

Example: Least squares

$$\min_{\boldsymbol{\theta}\in\mathbb{R}^d} \ \frac{1}{n}\|\boldsymbol{X}\boldsymbol{\theta} - \boldsymbol{y}\|_2^2 = \frac{1}{n}\sum_{i=1}^{n} \underbrace{(\boldsymbol{\theta}^\top \boldsymbol{x}_i - y_i)^2}_{\ell_i(\boldsymbol{\theta})}$$

- Each ℓ_i corresponds to one training data point $(\boldsymbol{x}_i, y_i)$.
- The above structure is called finite-sum and the problem is called finite-sum optimization.
- Finite-sum optimization is ubiquitous in machine learning, including most of the (supervised) learning problems.

Modern Large-Scale Machine Learning Problems

One of the key features of modern machine learning problems is large-scale, i.e., the number of training data n is very large (guided by VC analysis). Such a problem is called large-scale machine learning.

- One notable example is training NN.

We have the following two observations:

- The property we want for an algorithm is fast convergence. In terms of optimization, we should use fast algorithm (fast in terms of iteration) like first-order method (GD/AGD) or even second-order method (Newton).
- Guided by VC analysis, we just need to choose a fine $\mathcal{H}$ with small generalization error and then design a fast learning algorithm (GD/AGD or Newton) to learn $\widehat{\boldsymbol{\theta}}$ by solving the learning problem. Then, the learning process is done.

What is missed?

Modern Large-Scale Machine Learning Problems

One of the key features of modern machine learning problems is large-scale, i.e., the number of training data n is very large (guided by VC analysis). Such a problem is called large-scale machine learning.

- One notable example is training NN.

We have the following two observations:

- The property we want for an algorithm is fast convergence. In terms of optimization, we should use fast algorithm (fast in terms of iteration) like first-order method (GD/AGD) or even second-order method (Newton).
- Guided by VC analysis, we just need to choose a fine $\mathcal{H}$ with small generalization error and then design a fast learning algorithm (GD/AGD or Newton) to learn $\widehat{\boldsymbol{\theta}}$ by solving the learning problem. Then, the learning process is done.

What is missed? We do not consider **the computational issue** of the learning algorithm when n is large-scale.

Let us take GD as an example.

Gradient Descent in Large-scale Finite-sum Structure

GD:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \mu_k \nabla \mathcal{L}(\boldsymbol{\theta}_k)$$

- What is the structure of $\nabla \mathcal{L}(\boldsymbol{\theta}_k)$?

Gradient Descent in Large-scale Finite-sum Structure

GD:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \mu_k \nabla \mathcal{L}(\boldsymbol{\theta}_k)$$

- What is the structure of $\nabla \mathcal{L}(\boldsymbol{\theta}_k)$?

Due to the linearity of gradient operator, we have

$$\begin{aligned} \nabla \mathcal{L}(\boldsymbol{\theta}_k) &= \nabla \left(\frac{1}{n} \sum_{i=1}^{n} \ell_i(\boldsymbol{\theta}_k) \right) \\ &= \frac{1}{n} \sum_{i=1}^{n} \nabla \ell_i(\boldsymbol{\theta}_k) \end{aligned}$$

- Observation: The gradient of $\mathcal{L}$ is the summation of the gradients of all the component functions.
- It can be costly and even prohibitive to the compute the full gradient when n is very large.
- Newton's method is even worse when n is so large.